

Skill Without Training

Deterministic Knowledge Navigation for Large Language Models over Structured documents

Author: Yurii Chudinov

Affiliation: Independent Research & Software Solutions Architect

Keywords: LLM tooling, deterministic navigation, normative specifications, RFC 2119, ontology compilation, knowledge indexing, semantic enforcement, E.L.I.A

Patent status: Patent pending. Provisional patent applications covering the navigation architecture, index compilation method, and auto-regressive compiler described in this paper have been filed.

Abstract

Large Language Models often fail on large specifications not because they lack intelligence, but because they lack **deterministic navigation**.

A 500-page ISO-grade document contains deep hierarchical numbering, normative modality (MUST/SHOULD/MAY), cross-references across independent chapters, annexes with classification matrices—and no linear reading path. A human expert does not re-read the entire document for each question. They know where definitions live, where normative constraints are centralized, which annex encodes classification, and which section contains admissibility matrices.

Current approaches attempt to solve this by scaling context windows (up to 200k–300k tokens) or using embedding-based retrieval. However, even frontier models exhibit output instability, attention dilution (Liu et al., 2024), and reasoning variance when forced to process entire specifications in a single pass. Generic RAG retrieves semantically similar chunks but cannot distinguish definitions from examples or obligations from permissions.

This paper takes a different route: I encode the expert's mental map as a formal navigation algorithm. By precomputing structural indices, ontological routing rules, and normative relevance signals, I reduce the **navigation cost** to nearly zero, allowing the model to concentrate on reading and reasoning.

My system consists of a 3-pass ontology compiler that ingests a raw specification (e.g., 500 pages of markdown) and produces 14 machine-readable indices. At runtime, an MCP server uses these indices to answer queries via a deterministic pipeline: keyword enrichment → index search → ontological routing → reading plan construction → chain resolution → tier-weighted extraction.

The result is striking: an unmodified frontier model (tested on Claude 3.5 Sonnet through Claude Sonnet 4, reproduced on GPT-4) achieves 97–100% accuracy on a benchmark of 246 complex evaluation questions, with up to 15× fewer specification tokens delivered to the model and 7–10× faster responses than the agentic baseline.

No fine-tuning, no retraining—just structured tooling.

This work is part of a broader research project, the Auto-Regressive Compiler (ARC), which aims to stabilize inference in probabilistic systems. However, the core contribution here is the **index architecture**—how it

transforms a structured document into a deterministic reasoning substrate. The full index compiler and indices are open-sourced for reproducibility.

1. Introduction

LLMs fail on large specifications not because they lack intelligence, but because they lack **deterministic navigation**.

A 500-page ISO-grade document contains:

- Deep hierarchical numbering (e.g., 2.7.4.3.4.1)
- Normative modality (MUST / SHALL / SHOULD / MAY)
- Cross-references across independent chapters
- Annexes with classification matrices
- No linear reading path

A human expert does not re-read the entire document for each question. They know:

- Where definitions live
- Where normative constraints are centralized
- Which annex encodes classification
- Which section contains admissibility matrices

The system encodes this mental map as a formal navigation algorithm — conceptually aligned with the Theory of Constraints, which holds that system performance is determined by its primary bottleneck. In large-specification reasoning, that bottleneck is navigation rather than model intelligence. This assertion is supported not only by theoretical framing but by empirical evidence collected across iterative benchmark evaluations.

This algorithm constitutes a core component of my broader research project, the **Auto-Regressive Compiler (ARC)**. ARC is designed to stabilize inference in probabilistic systems and language models, achieving accuracy levels approaching 99.5% while significantly reducing operational overhead. It enables systematic expansion of AI capabilities without retraining or fine-tuning, by attaching formally defined logical structures —domains and modules—that constrain and guide inference deterministically.

Scope and nature of this work. This paper is part of a larger research program: the **E.L.I.A. language** (Enforcement-Level Integrated Architecture) and the **ARC compiler ecosystem** that processes it. The full system spans language design, semantic enforcement, compilation theory, and runtime stabilization of probabilistic models. This paper isolates and demonstrates one core principle: **deterministic navigation over structured documents eliminates the dominant cost factor in LLM specification reasoning**. The index architecture described here is the concrete mechanism through which a human expert's knowledge—encoded in a normative specification—is transferred to an LLM at inference time, without any weight modification.

This is engineering work. I do not propose a theoretical framework or prove a theorem; I build a system, deploy it against a real 500-page specification, and measure the outcome. The evaluation corpus—the E.L.I.A. specification itself—is **guaranteed not to appear in any LLM training corpus**: it is a private, unpublished language specification developed as part of this research, hosted in a private repository with no web-crawable presence. This eliminates data contamination risk entirely and ensures that every correct answer produced by the model is attributable to the navigation architecture, not to memorized training data. The

experiment therefore measures genuine capability transfer from structured document to inference-time behavior.

1.1 Motivation: Ontological Ambiguity in Specification Queries

A practical motivation for the index architecture emerged during the development of tools for reading large normative specifications such as GDPR, ISO standards, and the E.L.I.A. language specification itself. These documents (like my E.L.I.A. specification) frequently contain terms that are semantically overloaded: a concept may appear in many contexts, but the question often refers to a very specific ontological relation between concepts.

Consider the control question used during development:

"What is a delegate trust boundary and how is it enforced?"

Within the E.L.I.A. specification, both delegate and trust boundary exist as independent concepts. A naive retrieval system therefore tends to produce answers about delegates in general or about trust boundaries applied to multiple constructs. When token budgets are reduced during retrieval, the system may drift even further, producing responses that describe delegates broadly rather than the specific trust boundary relation attached to delegates.

The difficulty is not lexical; it is ontological. The query refers to a relationship between two concepts, not to either concept independently. Traditional retrieval mechanisms—whether full-text search or embedding similarity—lack a mechanism to prioritize this specific semantic relationship. As a result, the model often retrieves fragments that are individually relevant but collectively misaligned with the user's intent.

During early experiments, even a carefully engineered index system that performed well with a 12k token context window could exhibit drift when the retrieval budget was reduced. The answer would shift from "trust boundary of delegate" toward a generic description of "delegate". This revealed that the core problem was not search accuracy but ontological disambiguation under constrained context.

The WHAT | WHY | HOW | WHEN | WHERE classification introduced in this paper addresses this issue. By explicitly encoding the role of a concept within a question, the system can route queries toward indices that capture the correct semantic relationship rather than merely the correct vocabulary.

This paper therefore addresses a narrow but fundamental problem: enabling stable and reproducible answers to specification questions without requiring large inference budgets or probabilistic exploration of the document.

1.2 Core Hypothesis

Domain expertise in LLMs can be achieved without model training by encoding expert navigation strategy into deterministic tooling.

Formally:

```
Cost(answer) =  
  navigation_cost +  
  reading_cost +  
  reasoning_cost
```

For naive agents, navigation dominates: a substantial portion of tokens is spent determining which sections to consult, re-reading partially relevant fragments, backtracking across files, and iteratively refining search strategy. This overhead does not contribute to reasoning quality; it merely compensates for the absence of structured access.

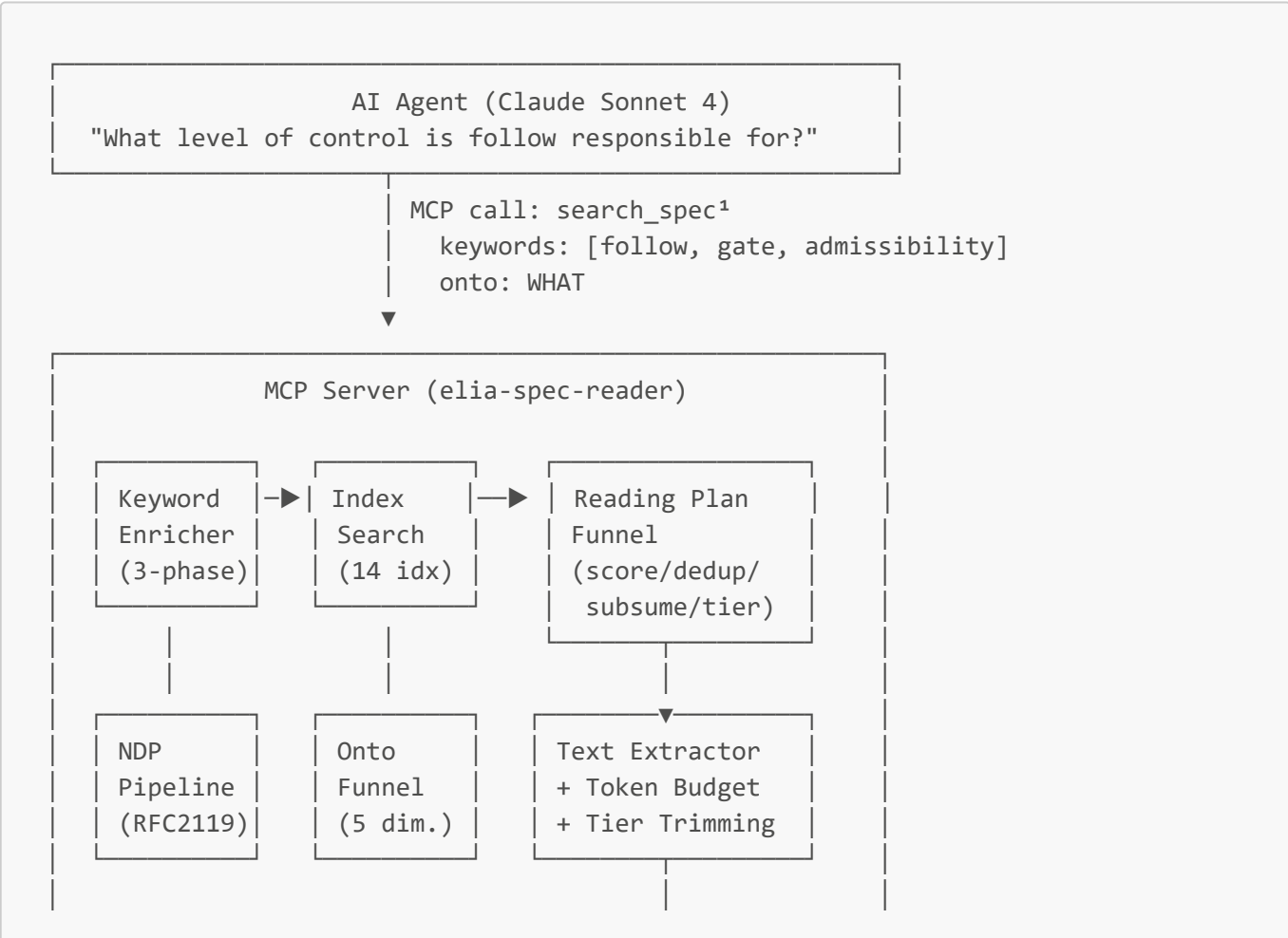
For domain experts, navigation cost approaches zero. The expert already possesses a mental index of the document—definitions, annexes, constraint matrices, and cross-references are accessed directly rather than rediscovered through trial and error.

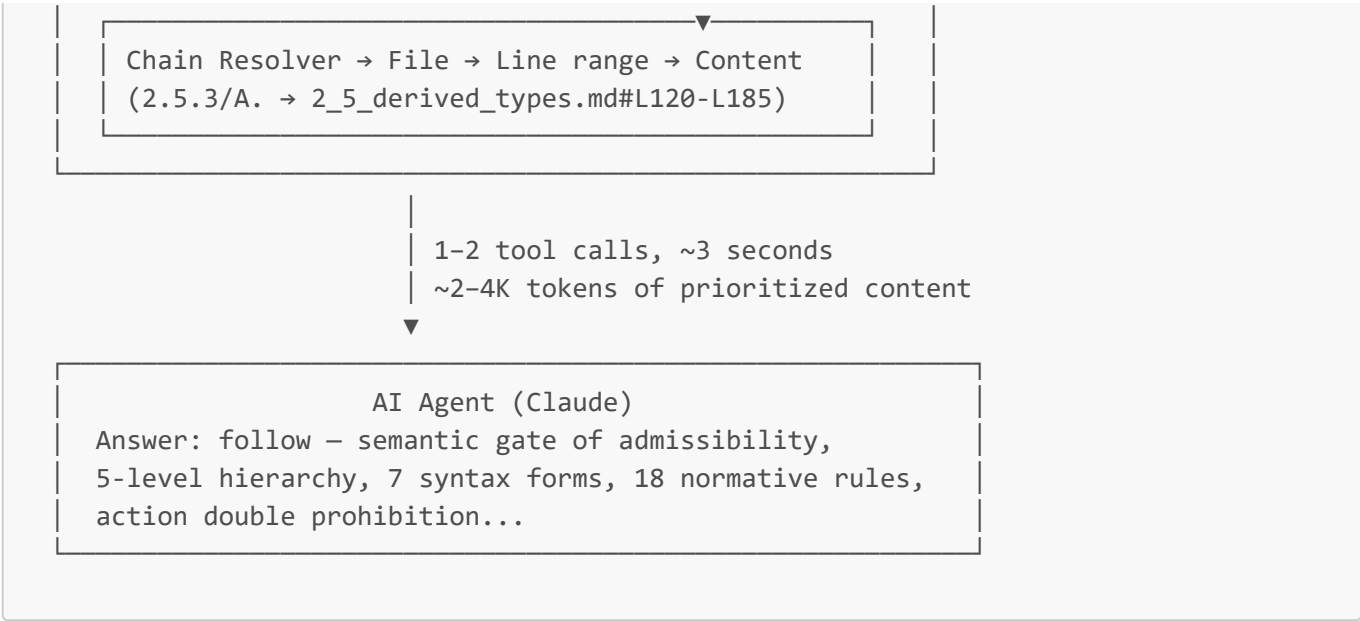
The architecture presented in this work operationalizes that expert-level navigation map. By precomputing structural indices, ontological routing rules, and normative relevance signals, the system reduces navigation cost to a negligible factor, allowing the model’s computation to be concentrated almost entirely on reading and reasoning.

2. Architecture Overview

Although this system is part of a broader Auto-Regressive Compiler (ARC) research program, the primary contribution of this article is the **index system** operated by the compiler. The 3-pass pipeline is important context, but the central technical innovation lies in how structured indices transform a large normative specification into a deterministic, navigable reasoning substrate.

In other words, this paper is about the index architecture: how indices are constructed, how they interact, how they constrain search space, and how they enable stable reasoning using only full-text search plus deterministic routing.





¹ The MCP server (v0.3.0, TypeScript, ~4,850 lines) exposes 7 tools: `search_spec` (keyword → reading plan → content), `read_chain` (direct chain address extraction), `list_units` (index discovery), `fulltext_search` (raw grep), `lookup_xref` (cross-reference traversal), `analyze_question` (query decomposition and classification), and `spec_version` (metadata). The diagram shows the primary query path through `search_spec`.

2.0 Goal

The objective is to build a pipeline that takes a formal specification (or any normative document) and automatically produces:

- 1. **Structured ontology** — concepts, relationships, and constraints extracted from canonical document structure
- 2. **Deterministic step-by-step instructions** — precise reading sequences, decision trees, and navigation maps that an LLM can follow without hallucination
- 3. **Self-validating prompts** — instructions that carry their own correctness criteria, derived from normative modality (MUST/SHOULD/MAY) in the source

2.1 The 3-Pass Context Analyzer

Pass	Input	Output	Action
Pass 1 — Structure	Raw markdown (≈500 pages)	Chain addresses, section graph, cross-references, vocabulary	Built (linear index + xref)
Pass 2 — Semantics	Structured sections	Aspect indices, ontological roles, keyword maps, Terms & Definitions terms	Built (13 aspect indices)
Pass 3 — Normative	Semantic units	MUST/SHOULD/MAY extraction, enforcement graph, admissibility rules	Runtime (NDP pipeline)

Implementation note. The "3-pass compiler" is a conceptual pipeline implemented as a collection of build scripts and runtime modules, not a monolithic binary. Passes 1–2 run at build time (producing the 14 index artifacts); Pass 3 (NDP) runs at **query time** inside the MCP server, scoring retrieved fragments by normative weight.

2.2 Core Algorithm: definition as the Primary Determinant of Meaning

The current algorithm is grounded in the E.L.I.A. definition construct — the specification’s own mechanism for fixing meaning:

```
definition:
  name: "<concept>"
  explained as: "<the meaning – the only source of semantic truth>"
  enforced: [<aspects where this meaning applies>]
  constraints:
    MUST: "<non-negotiable rules>"
    SHOULD: "<recommended practices>"
    MAY: "<optional behaviors>"
```

3. ISO-Grade Structure as a Machine-Readable Coordinate System

ISO/IEC, ENISA, DORA, RFC standards share structural invariants:

- Deep hierarchical numbering (multi-level section addresses up to 5–6 depths)
- Lettered and enumerated paragraphs enabling fine-grained extraction
- Explicit cross-referencing chains across chapters and annexes
- RFC 2119 modality (MUST / SHALL / SHOULD / MAY) with enforceable priority semantics
- Annex-based classification matrices (normative classes, admissibility tables, lifecycle rules)
- Separation of semantic, structural, and behavioral layers within the document
- Explicit enforcement and admissibility constructs (definition → enforcement → constraint)
- Deterministic traceability between main sections and annexes
- Structured decomposition of obligations, roles, and domain boundaries

These conventions form a latent coordinate system.

Example mapping:

```
§2.7.4           → "2.7.4"
§2.7.4/A.        → "2.7.4/A."
§2.7.4/(A)-(F)   → range extraction
§2.7.4/all       → recursive extraction
§2.7.4/table-1   → structured table
```

I did not invent this coordinate system; I operationalized it. The architecture diagram in Section 2 illustrates the MCP server’s **reasoning and retrieval pipeline architecture** used to produce correct answers using only **full-text search** plus the **index system** (i.e., structured retrieval and deterministic reading-plan construction), rather than embedding-based similarity retrieval.

4. Index Architecture — The Index System Operated by the Compiler

This section is the core technical contribution: a multi-layer index architecture that turns a 500-page ISO-grade specification into an addressable, navigable, and budget-controllable reasoning substrate. The system

uses **14 index artifacts** organized into **four complementary architectures**, each solving a distinct navigation problem.

4.0 Why indexing works in ISO-grade documents

Normative specifications already contain a latent coordinate system: deep section numbering, lettered paragraphs, tables, and explicit cross-references. The compiler does not “infer” meaning from free text; it compiles declared structure into indices.

Two ideas are central:

- 1. **Addressability**: every relevant unit of knowledge has a stable coordinate.
- 2. **Traceability**: cross-references form a directed graph that mirrors audit practice.

4.1 Chain Address System

The **chain address** is the universal coordinate used by all indices. It encodes what to read and how precisely to read it—down to a single lettered paragraph or a table—without loading entire files.

Address forms (illustrative examples):

- 2.5.3 — section heading + first paragraph
- 2.5.3/A. — a specific lettered paragraph
- 2.5.3/(A)-(F) — a paragraph range
- 2.5.3/all — full subtree extraction (section + all subsections)
- 2.5.3/table-1 — a structured table within the section

Operationally, a chain address is resolved through the linear backbone into **(file, line-span, node-type)**, enabling single-pass extraction.

All indices are declared via a centralized registry file (`aspect_index.json`) that binds each index to an alias, file path, default ontological affinity, and behavioral profile. This registry is part of a broader **root specification descriptor** (`specification_struct.json`) that declares the document type, specification version, a pointer to the structure map, the unit-structure contract (how `unitName`, `keywords`, `seq`, `$ref`, and heading normalization are interpreted), and the search-protocol pipeline stages. Together, the root descriptor and registry form the reusable portability surface: adapting the system to a different specification requires updating these declarative files, not the MCP server logic.

4.2 Four Index Architectures

The compiler outputs 14 indices grouped into four architectures:

Architecture	Indices	What it solves	Core schema
Unit-based	11	Concept routing: "what to read for this concept?"	<code>unit</code> → { <code>keywords</code> [], <code>seq</code> []}
Graph-based	1 (<code>xref</code>)	Discovery: "what else is linked to this section?"	<code>section</code> → <code>refs</code> []

Architecture	Indices	What it solves	Core schema
Linear / inverted	1 (linear)	Physical resolution + reverse lookup + vocabulary	sections, reverseMap, termIndex
Structural	1 (grix)	Admissibility: "what is allowed/forbidden where?"	block → admits/forbidden

The unit-based set includes 11 indices (by aspect): `phya`, `sema`, `ont`, `desa`, `grma`, `bhva`, `phla`, `trma`, `onma`, `bsyn`, `grix`.

Note: `grix` appears in both **Unit-based** (it contains keyword-searchable units) and **Structural** (it encodes admissibility matrices) architectures. The total of 14 unique index artifacts remains unchanged.

The 11 unit-based indices can be viewed as instantiations of **6 abstract index roles** that any domain-adapted deployment can use:

- `onto` — foundational purpose and existence rationale
- `desa` — strategic realization and design concepts
- `trma` — normalized cross-document term-definition mapping
- `aspect` (1–N domain-specific) — bounded thematic views (WHAT/WHEN, WHY/WHEN, etc.)
- `onma` — semantic role binding (concept × question-type)
- `physa` — physical realization and operational controls

In the E.L.I.A. domain, the `aspect` role is instantiated 6 times (`phya`, `sema`, `bhva`, `phla`, `grma`, `bsyn`), yielding 11 unit-based indices containing a combined **396 units** across all aspect, ontological, and terminology indices.

Note on `phya` vs `physa`: The E.L.I.A. `phya` index captures physical type-system semantics (data types, serialization, materialization). The abstract `physa` role in the generalized framework maps to concrete deployment artifacts — evidentiary objects, operational controls, technical mechanisms. In the E.L.I.A. domain, `phya` serves both roles because the specification is simultaneously the subject and the implementation target.

4.3 Unit-based indices: “concept → reading program”

A **unit** is not just a pointer; it is a **reading program** that encodes expert order-of-consumption (definition → constraints → annexes/examples).

```
{
  "delegate": {
    "keywords": ["delegate", "boundary", "cross-border", "trust"],
    "seq": [
      {"Order": 1, "Tier": 1, "Intent": "semantic_role", "chain": ["2.7.4"],
      "Note": "Definition"},
      {"Order": 2, "Tier": 2, "Intent": "normative_rules", "chain":
["2.7.4.3/all"], "Note": "Constraints"},
      {"Order": 3, "Tier": 3, "chain": ["A.7"], "Note": "Registry entry"}
    ],
    "abstract": "A delegate represents a boundary-crossing invocation..."
  }
}
```


Field semantics (most used across unit indices):

- **keywords**[]: matching surface for query and enrichment
- **seq**[]: ordered reading steps
- **Order**: strict priority inside the unit
- **Tier** (1–4): budget trimming priority (drop T4 first under pressure)
- **Intent**: reading-mode classification (definition, constraints, rationale, syntax, etc.)
- **Note**: human-readable justification
- **abstract**: optional per-unit summary

4.3.1 Schema diversity across unit indices

Not all unit indices need the same schema richness. The ontological routing index (**onma**) is intentionally rich (it is the hub), while pure grammar indices can be minimal.

4.3.2 Semantic coupling pairs: why they make navigation deterministic

A central innovation of the index architecture is the use of **semantic coupling pairs** — explicit multi-role ontological bindings at both the index and unit level. Each aspect index declares a *default ontological affinity* (e.g., **phya** → WHAT, **bhva** → HOW+WHEN+WHERE), and individual units within an index can override or refine it.

This two-level scheme is what makes routing **deterministic rather than heuristic**:

1. **Index-level affinity** acts as a coarse filter. When the ontology funnel classifies a query as WHEN, it knows that **bhva** (default: HOW+WHEN+WHERE) is a candidate index but **phya** (default: WHAT) is not — without scanning a single unit.
2. **Unit-level coupling pairs** provide fine-grained routing within an index. A unit marked **HOW+WHEN** answers "how does X work and when does it trigger?" in a single reading plan, whereas a neighbouring unit in the same index marked **WHAT** answers "what is X?" — same index, different route.

In the current E.L.I.A. implementation, the system contains **17 unit-level coupling pairs** distributed across 5 indices:

Coupling pair	Count	Where (indices)
HOW × WHEN	5	sema , ont , desa , bhva
WHAT × HOW	5	ont , bhva
WHY × HOW	3	ont
WHEN × WHERE	3	onma
HOW × WHERE	1	onma

Additionally, 5 of 11 indices carry **multi-role defaults** at the index level: **ont** (WHY+WHAT), **desa** (HOW+WHY+WHERE), **bhva** (HOW+WHEN+WHERE), **onma** (all 5 roles), **grix** (HOW+WHAT). The remaining 6 indices have single-role defaults.

Why this matters: without coupling pairs, a query "when should I use a compilation phase model?" would require the system to speculatively search multiple indices. With the explicit **HOW+WHEN** pair on **bhva/compilationPhaseModel**, the ontology funnel routes directly — zero speculative search, deterministic chain selection. The same mechanism enables a single unit to serve compound questions ("what is X and how does it work?") without duplicating content across indices.

The generalized framework formalizes these as a required property of aspect indices: each aspect must declare which ontological roles it covers, as pairs or triples (WHAT/WHEN, WHY/WHEN, HOW/WHEN, WHERE/WHY, etc.), so that any domain-adapted deployment inherits the same deterministic routing.

4.4 **onma**: Ontological Role Index (concept × question-type matrix)

onma does not merely index concepts; it indexes **question-roles about concepts** using a naming scheme like **<concept>_<role>**.

Examples:

- **typeSystem_what** → definition sections
- **typeSystem_why** → rationale sections
- **typeSystem_how** → mechanics + annexes
- **record_what** / **record_why** / **record_how** → different routing for the same concept

This enables deterministic routing: the same keyword ("record") maps to different reading plans depending on WHAT/WHY/HOW/WHEN/WHERE intent. The current **onma** index contains **69 units** across 22 constructs.

4.5 **trma**: Terminology Index (Terms & Definitions accelerator)

trma indexes defined terms and routes to Terms & Definitions tables. It is used primarily for **query understanding** (keyword enrichment), not answering. Units typically have **Intent: definition** and point to a T&D table chain (e.g., **1.0/table-1**).

This is the first stage of enrichment: translating user vocabulary into spec-native terminology.

4.6 **xref**: Cross-reference graph (forward + reverse traversal)

Cross-references ("see §...", "as defined in §...") are compiled into a directed graph:

- **166 sections** indexed
- **291 cross-references** compiled
- edge types: **section** (internal), **standard** (ISO/RFC/GDPR), **url** (external)

Each node stores the origin section and a typed list of outbound edges with context. Reverse lookup enables "who references GDPR?" style queries.

4.7 **linear**: The backbone index (physical resolution + inverted vocabulary)

The **linear** index is the physical substrate that makes chain addresses executable. It combines four structures into one artifact (≈95k lines):

- **sections** (~1,211): every heading with (file, title, level, line number, vocab[])
- **reverseMap** (~1,145): chain address → which units reference it (reverse navigation)

- **termIndex** (~7,198): inverted vocabulary → candidate section addresses
- **meta**: generation stats

Key property: **linear** bridges **logical addressing** (2.5.3/A.) to **physical extraction** (file + line-range).

4.8 **grix**: Grammar instruction index (admissibility matrices)

grix is not a search index. It is a structural correctness map that answers WHERE-type questions ("is X admissible inside Y?") in constant time.

For each block type (≈ 8), it encodes:

- canonical syntax
- spec anchors
- examples
- admitted constructs/declarations/operators
- forbidden constructs

This makes admissibility machine-queryable and supports validation logic.

4.9 **structure_map.json**: Portable document grammar (adapter layer)

In addition to the 14 indices, the system uses a portable structural adapter: **structure_map.json**. This is a formal grammar of ISO/EU-style documents (document vs annex canvas, H1–H6 sections, paragraphs, lettered sequences, lists, tables, and typed references). It is consumed by the **agent instruction layer** (Copilot prompt configuration), where it guides how the AI agent interprets document structure during query processing. The MCP server itself does not import **structure_map.json** at runtime; it relies on the pre-compiled **linear** index for physical resolution.

The portability mechanism is explicit:

- **patterns.json** captures regex patterns for heading detection, chain derivation, and reference parsing.
- **structure_map.json** defines node types and recursive composition.

Adapting the system to a different ISO-grade document is primarily a matter of updating these adapters, not rewriting index logic.

4.10 How indices compose during query answering

At runtime, indices are composed into a deterministic pipeline:

1. **Keyword enrichment** (T&D → co-occurrence → waterfall) to align user vocabulary with spec vocabulary
2. **Index search** across unit-based indices (strong vs weak keyword weighting)
3. **Ontological routing** via **onma** (WHAT/WHY/HOW/WHEN/WHERE)
4. **Graph expansion** using **xref** (optional, discovery)
5. **Reading-plan construction** over chain addresses (deduplication + subsumption + grouping)
6. **Resolution and extraction** using **linear**
7. **Budget enforcement** using Tier trimming

This index-first design is what makes “full-text search + indices” competitive with (and, for normative specs, more stable than) naive context-scaling or generic embedding-based retrieval.

4.11 Index Artifacts Inventory

For clarity, the full index system produced by the compiler can be viewed as a coordinated inventory of artifacts rather than a single retrieval layer.

Index	Type	Purpose
phya	Unit	Physical structure semantics
sema	Unit	Meaning-level routing
ont	Unit	Ontological concept grouping
desa	Unit	Design rationale access
grma	Unit	Grammar-oriented navigation
bhva	Unit	Behavioral mechanics
phla	Unit	Philosophical foundations
trma	Unit	Terminology alignment
onma	Unit	Ontological role routing
bsyn	Unit	Base syntax support
xref	Graph	Cross-reference discovery
linear	Backbone	Physical resolution + inverted vocabulary
grix	Structural	Admissibility logic
structure_map	Adapter	Portable document grammar

The indices divide into two maintenance classes: **7 curator-maintained** (phya, sema, ont, desa, bhva, phla, bsyn) are manually curated by a domain expert, while **4 are pipeline-generated** (onma, grma, trma, grix) and can be rebuilt deterministically from document structure. The auxiliary indices (linear, xref) are always auto-generated. This separation enables quality control: curated indices encode expert judgments that cannot be mechanically inferred, while generated indices are reproducible at any time.

Each artifact is deliberately scoped. No index attempts to “do everything”. Stability emerges from composability.

4.12 Reverse Navigation and Explainability

One of the less obvious but critical properties of the system is **reverse navigation**.

The `reverseMap` component of the `linear` index allows the MCP server to answer not only:

“What should I read to answer this question?”

but also:

"Why am I reading this section?"

Because each chain address can be traced back to the unit(s) that selected it, the system can reconstruct its own reasoning path:

- Which concept triggered retrieval
- Which ontological role applied
- Which index contributed the chain

This provides a foundation for explainable navigation — a prerequisite for auditability in regulatory environments.

In practice, reverse navigation enables:

- deterministic justification
- traceable reading plans
- rejection of irrelevant context

This property is particularly important when operating over compliance-grade specifications, where provenance of interpretation matters as much as correctness of outcome.

5. Ontological Funnel

Questions are classified by ontological role:

- **WHAT — identity**
Retrieval prioritizes **definitions and type declarations**. The index system first queries `onma` for `<concept>_what`, then loads Tier-1 chains (usually definition sections) and resolves them through `linear`.
- **WHY — rationale**
Retrieval targets **design explanations and conceptual motivations**. The system searches unit indices such as `desa` and `ont`, selecting chains whose `Intent = rationale` before resolving them via `linear`.
- **HOW — mechanics**
Retrieval focuses on **operational behavior, grammar rules, and execution procedures**. The index search prioritizes `grma`, `bhva`, and `phla` units, constructing a reading plan from their `seq` fields.
- **WHEN — conditions**
Retrieval selects **normative constraints and lifecycle conditions**. The pipeline combines `onma` routing with the **Normative-Driven Pipeline (NDP)** so sections containing MUST/SHOULD logic receive higher priority.
- **WHERE — admissibility**
Retrieval resolves **structural placement rules**. The system consults the `grix` structural index to determine admissibility matrices and then extracts the referenced chains through the `linear` resolver.

The same keyword routes differently depending on role.

Example:

"rule"

- WHAT → type definition
Algorithm: `onma` is queried for `<concept>_what`. Returned chains typically point to definition sections (Tier-1). Chains are resolved through `linear` to obtain exact file and line spans.
- HOW → grammar + body syntax
Algorithm: keyword matches are searched across `grma`, `bhva`, and `phla` unit indices. The system merges their `seq` lists, sorts by `(Tier, Order)`, and builds a reading plan. Chains are then resolved via `linear`.
- WHY → design rationale
Algorithm: the system searches `desa` and `ont` indices for entries with `Intent = rationale`. Selected chains are prioritized if referenced by `xref`, then resolved through `linear`.
- WHERE → constraint matrices
Algorithm: admissibility queries trigger the `grix` structural index. If a matrix references specification sections, those chains are passed to `linear` for extraction.

This single parameter eliminates cross-domain noise.

6. Normative-Driven Pipeline (NDP)

RFC 2119 keywords are treated as machine-readable signals.

Extraction pipeline:

- **MUST / SHALL → enrich (high weight)**
Treated as hard obligations. Sections/paragraphs containing these markers receive a priority boost during reading-plan scoring and are promoted to Tier-1/2 where possible.
- **SHOULD → enrich (medium weight)**
Treated as recommended constraints. Boosted, but typically ranked below MUST/SHALL when competing for token budget.
- **MAY → conditional (low weight)**
Treated as optional behavior. Included when (a) the query explicitly asks for options/variants, or (b) higher-tier material is insufficient.
- **MUST NOT / SHALL NOT → filter + elevate as a prohibition**
Two effects: (1) if a candidate fragment is only tangentially relevant, it can be dropped (filter); (2) if the question concerns admissibility or safety, prohibitions are promoted because they are high-signal constraints.

How the filter works operationally

1. **Extract modality signals** per paragraph (and per table row where possible).
2. Assign a **normative score** (e.g., MUST=3, SHOULD=2, MAY=1, NOT adds +2 and a prohibition flag).
3. Feed this score into the reading-plan funnel as an additive feature ("NDP boost").
4. Under budget pressure, keep higher normative scores and drop lower ones first.

Extending NDP when normative markers are missing

When a document does not consistently use RFC-2119 vocabulary, NDP can be extended with a fallback “deontic and constraint cue” layer:

- **Deontic verbs:** *required, prohibited, permitted, obliged, responsible for, ensure, enforce, must be able to.*
- **Policy/controls phrasing:** *controls include, shall implement, is responsible for, approval required, review frequency, retention period, audit trail.*
- **Constraint templates:** *only if, unless, except, no later than, at least, at most, within X days, prior to, after, before.*
- **Negation patterns:** *not allowed, forbidden, cannot, may not, never.*
- **Structural priors:** headings like *Requirements, Controls, Compliance, Obligations, Prohibitions, Exceptions*, plus annex matrices.

In implementation terms, this is simply an additional signal extractor that populates the same fields as RFC-2119 (obligation/recommendation/option/prohibition), so the downstream ranking and Tier trimming logic remains unchanged.

This converts normative language into a deterministic relevance layer.

Meaning is not guessed.

It is derived.

7. Weighted Reading Plan Funnel

Reading plan scoring:

The reading plan is ranked using a deterministic scoring function. Each candidate chain produced by the indices carries metadata that influences priority. The scoring function converts these metadata signals into a single **effective priority score** used by the funnel.

Field definitions

- **metaWeight** — semantic importance of the unit that produced the chain.
This value is defined in the index itself and reflects how critical the fragment is for understanding the concept. Typical examples:
 - definition sections
 - normative constraints
 - admissibility matrices
 - registry tables

High-signal units (definitions, constraints) receive larger **metaWeight** values so they dominate the reading plan even if many weaker matches exist.

- **order** — the explicit reading order defined in the unit's **seq[]** program.
This encodes expert reading strategy. For example:

1 → definition
2 → normative constraints
3 → examples or annex material

Lower order numbers therefore receive higher priority during ranking.

- **ontoMatch** — ontological role *mismatch* score between the query role and the candidate chain. A value of 0 indicates perfect alignment (e.g., a WHAT question retrieving a definition), while larger values indicate increasing semantic mismatch. The value therefore acts as a penalty term in ranking. If the user's question was classified as WHAT, chains whose intent corresponds to definitions receive the strongest match. If the role is HOW, grammar and behavioral sections are favored. Because the value represents mismatch, higher values reduce the final score in the ranking formula, ensuring semantically irrelevant material is suppressed.
- **crossRefPenalty / boost** — influence from the xref graph. Chains that appear frequently as cross-reference targets are considered authoritative anchors and may receive a boost. Conversely, weakly connected fragments may receive a small penalty.
- **demotions** — penalties applied during funnel processing. These include duplicate chains, parent-child overlap, or fragments exceeding token budget constraints.

Together these signals produce a deterministic ranking score:

```
effectivePriority =
  metaWeight × 30
+ order × 10
- ontoMatch × 15
+ crossRefPenalty
+ demotions
```

Then:

1. Deduplicate

Chains referring to the exact same address (e.g., 2.7.4 repeated by several indices) are collapsed into a single candidate. Their metadata signals (metaWeight, ontoMatch, crossRef) are **summed or merged**, preserving the strongest priority values.

2. Subsume parent/child chains

If both a parent section and one of its subranges appear (e.g., 2.7.4/all and 2.7.4/A.), the system keeps only the **most specific fragment** unless the parent contains additional paragraphs not covered by the child. This prevents redundant extraction while preserving information coverage.

3. Group by file

All remaining chains are grouped by the source file resolved via the linear index. This allows the extractor to compute minimal line spans and avoid opening the same file multiple times.

4. Extract once

Within each file group, overlapping line ranges are **merged into a single extraction span**. The file is read only once and the required fragments are sliced from the resulting buffer. This minimizes I/O and token duplication.

5. Apply tier-aware trimming

Each chain carries a Tier value from the unit index (1 = critical, 4 = optional). If the total token estimate exceeds the allowed budget, the funnel progressively removes candidates starting from **Tier-4**, then

Tier-3, while preserving Tier-1 and Tier-2 material. Within the same tier, the previously computed **effectivePriority** score determines which fragments survive.

The budget system (implemented in ~460 lines of dedicated logic) supports four **detail levels** that control which tiers are protected from trimming:

Detail level	Protected tiers	Use case
brief	T1	Quick lookups — definitions only
normal	T1 + T2	Default — definitions + normative properties
detailed	T1 + T2 + T3	Deep queries — + registry, cross-refs, matrices
complete	all	Full extraction, no trimming

When trimming would drop **protected** content, the server emits a budget-pressure signal and can **auto-expand** the token window (ceiling: 20,000 tokens) to preserve critical material. This ensures that the model always receives complete definitions and normative constraints, even under tight budgets.

Navigation overhead becomes zero.

8. Results

Before introducing the index architecture described in this paper, direct LLM processing of the specification was tested using repeated queries over the full document. Without the index algorithm, **accuracy did not exceed ~85% (ontological drift and hallucination observed)**, and answers were unstable: repeating the same question often produced different results due to probabilistic navigation across the specification. In these baseline runs the model attempted to reason by scanning large portions of the document, typically consuming **62k–190k tokens** per full pass depending on search strategy and context expansion.

Corpus validity. The E.L.I.A. specification used for evaluation is **guaranteed not to appear in any LLM training corpus**. It is a private, unpublished language specification developed as part of this research, hosted in a private repository with no web-crawable presence. This eliminates data contamination risk and ensures that model accuracy is attributable solely to the navigation architecture.

8.1 Accuracy

Stage	Accuracy	Tokens Used
Baseline (full-text search)	~70%	~24k
After indices	~87%	~10k
After cross-ref	~95%	~6k
After tier weighting	100%	~3.8k
After NDP + line filter	100%	~2k (range 1.5–3.5k)

246 evaluation questions passing (run 5× each for stability). Note: the last two stages both achieve 100% accuracy; NDP + line-level filtering reduces token cost, not accuracy.

The system is validated by 338 unit tests (Vitest) covering the MCP server internals and 246 evaluation questions covering end-to-end accuracy.

8.2 Cost

Complex query comparison:

Metric	MCP	Baseline
Tool calls	1–2	15–20
Wall time	~3s	25–35s
Tokens	~2k	25–30k
Navigation overhead	~0%	~40%

The system reads more relevant content but reasons less.

Two distinct baselines yield two different efficiency ratios:

- **Agentic baseline** (grep + read_file loop, ~25–30k tokens per question): MCP uses ~2–4k tokens → **7× reduction**. This is the comparison in the table above and in the abstract.
- **Full-context baseline** (entire 178k-token spec cached per question): MCP extracts on average ~11.6k tokens → **15× reduction**. This measures raw spec-content delivery, not total API tokens.

The 7× figure reflects end-to-end operational cost (tool calls, navigation overhead, reasoning). The 15× figure isolates the retrieval layer: how much specification text the model must process. Both confirm that structural indices eliminate the dominant cost factor — navigation.

The MCP extraction range was 2k–20k tokens depending on query complexity and `autoExpand` behavior (9 out of 10 measured queries auto-expanded from the initial 3k budget; the ceiling is 20k). Only simple single-concept queries fit within the default budget; compound or cross-referential questions required 7–20k tokens of targeted content.

Accuracy under full-context loading. Even when the entire specification fits inside the model's context window (~178k tokens within a 200k limit), accuracy does not converge to 100%. In the hard-36 subset (questions specifically selected for cross-referential and non-obvious reasoning), the naive full-context baseline failed to answer **28% of questions** (10/36 UNABLE) — the model explicitly acknowledged it could not locate the relevant content. Category D (design rationale) was worst at 43% failure rate, because design justifications are scattered across multiple non-adjacent sections that the model cannot reliably cross-reference by scanning alone.

This demonstrates a fundamental limitation: **loading a document into context is not the same as understanding its structure**. Large-context models exhibit *attention dilution* (Liu et al., 2024) — as the haystack grows, the probability of correctly attending to the specific paragraphs that answer a compound question decreases. The MCP pipeline avoids this entirely: by extracting only the 2–20k tokens that the index system identifies as relevant, the model's attention is concentrated on precisely the right fragments. In the MCP conceptual benchmark (10 questions, expert-scored), factual accuracy was **100%** (10/10, accuracy=3/3) with a mean quality score of **0.967**, and zero UNABLE answers.

8.3 Benchmark Design

To evaluate the system, I constructed a test suite of 246 questions over the E.L.I.A. specification. The questions were categorized into four types:

- **Factual (82 questions):** Direct lookups (e.g., "What is the syntax of a record declaration?")
- **Cross-referential (74 questions):** Requiring traversal of links between sections (e.g., "How does the delegate rule in §2.7.4 interact with Annex A.7?")
- **Philosophical/Abstract (45 questions):** No direct keyword match (e.g., "record immutability" — "immutability" does not appear in any heading).
- **Normative (45 questions):** Requiring interpretation of MUST/SHOULD/MAY (e.g., "Is it allowed to omit the 'enforced' clause?")

Each question was run 5 times to measure stability. Baseline used a naive agent consisting of a standard LLM tool loop (Claude 3.5 Sonnet) with two tools: `grep_search` (ripgrep-based pattern search) and `read_file` (line-range extraction). The agent operated in an agentic loop with a $\approx 200\text{k}$ -token context window, autonomously deciding which sections to search, load, and re-read. It had no structural indices, no pre-computed reading plans, and no deterministic navigation guidance — replicating the typical "LLM + file tools" setup used in practice. The same 246 questions were used for both baseline and MCP runs; baseline accuracy was measured per-question as pass/fail by the same human evaluator. The full evaluation protocol — including the 4-artifact per-question format (plan/render/final/bench), scoring rubric, runner infrastructure, and multi-model results — is described in **ANNEX B**.

The §8.1 table shows **progressive improvement** within the MCP pipeline: the $\sim 70\%$ "Baseline" row corresponds to using indices for keyword matching only — without cross-reference expansion, tier weighting, or NDP scoring. The $\sim 85\%$ ceiling mentioned above refers to the separate naive-agent baseline (no indices at all), which exhibited high variance across repeated runs.

8.4 Reproducibility

The benchmark dataset (246 questions, English) is included in the open-source repository at `tools/copilot-2/benchmark_questions_246_en.txt`. An additional hard subset (36 questions targeting non-obvious, cross-referential, and philosophical queries) is at `tools/copilot-2/benchmark_hard_37_en.txt`. The MCP server (v0.3.0), all 14 index artifacts, and the build scripts are included in the same repository. To reproduce:

1. Build indices: `node tools/spec-reader/src/scripts/buildAspectIndices.ts`
2. Start MCP server: `node tools/spec-reader/dist/server.js`
3. Run benchmark: configure an MCP-compatible agent with the 7 exposed tools and submit the question file

No embedding model, no vector database, and no external API beyond the LLM itself is required.

9. Deterministic Inference Without Training

9.1 The Problem of Non-Obvious Queries

Any realistic benchmark contains questions whose answers are not "on the surface". In such cases there are **no direct keyword matches** in headings—and often none in the local text around the correct answer either. This is a test of whether the system understands (and can traverse) the specification's structure, rather than merely performing dictionary-style lookup.

In large normative documents, this is common:

- the *concept* is defined under a different term
- the *property* is encoded as an admissibility rule or a metadata constraint
- the binding logic is placed in an annex (classification matrix, lifecycle rules, registry)

The purpose of this section is to show that the system reaches these “hidden” answers through deterministic index-driven navigation.

9.2 Case study: “record immutability”

Raw query: `record immutability`

Observation: the word “immutability” does not appear as a heading (and may not appear anywhere in the specification verbatim). A naive LLM approach tends to drift: it will search broadly, read large spans of the document, and produce variable answers across repeated runs.

Below is the **reconstructed** processing trace — a representative walkthrough of how the pipeline would process this query, based on the system's actual algorithms and index structures. Exact enrichment terms and chain addresses depend on the current index state at query time. **ANNEX A** contains real, unedited MCP server traces (`verbose: debug`) for this and two additional queries.

Step 1 — Keyword enrichment via terminology index (`trma`)

The query is first normalized into **spec-native vocabulary**. The terminology index (`trma`) is used to map the user’s phrasing onto internal anchors:

- `record` → routes to the record construct as a block/derived type
- the property “immutability” is not searched as a literal token; instead it is treated as a **constraint-like attribute**, which in E.L.I.A. is typically expressed through metadata flags and lifecycle/admissibility rules

This yields an enriched keyword set similar to:

- `record, block type, derived type, metadata, flags, constraints, admissibility`

(Exact enrichment output depends on the T&D entries and co-occurrence maps extracted during Pass 2.)

Step 2 — Unit-index search (`onma`, `grma`, optional `bhva`, depending on WHAT/HOW/WHERE role)

Using the enriched keywords, the server searches unit indices:

- `onma` to find concept-role routes (e.g., `record_what`, `record_how`)
- `grma` to locate grammar/placement rules relevant to record declarations
- optionally `bhva` if the system detects that the question concerns behavior/state transitions rather than pure syntax

Each matched unit yields one or more chain candidates from its `seq[ ]` reading program.

Step 3 — Ontological routing (role selection)

The query is classified into an ontological role (or a small role set). For “record immutability” the typical outcomes are:

- **WHAT:** “what is immutability of record in this system?” → prioritize definition + normative constraints
- **HOW / WHERE:** “how is record immutability achieved / where is it enforced?” → prioritize admissibility, metadata flags, and annex matrices

This role selection determines which **onma** entries dominate and which unit indices (**grma**, **bhva**, **grix**) are prioritized via **ontoMatch** in the funnel.

Step 4 — Reading plan funnel + chain resolution (**linear**)

All candidate chains are merged and ranked by the funnel:

- deduplicate identical chains (sum metadata signals)
- subsume parent/child overlaps (prefer the most specific fragment)
- group by file (minimize file reads)
- extract once per merged span
- apply tier-aware trimming under budget pressure

Finally, chain addresses are resolved through **linear**:

- **chain** → **file** → **line range** → extracted content

Step 5 — Result: correct annex found without lexical match

The system reaches the correct fragment by following the compiled topology:

record → **derived type** → **metadata flags** → **annex / matrix defining immutability constraints**

Even though “immutability” does not appear as a literal keyword, the relevant rule is reachable through deterministic structural dependencies (definitions → constraints → annex classification).

9.3 Additional non-obvious examples (brief)

Two additional benchmark cases illustrate the same deterministic navigation pattern.

- **Governance acceptance** — a query about governance acceptance of a construct. The answer is not stated in a single section but emerges from the registry structure in an annex. The navigation path follows: concept → governance semantics → annex registry entry → cross-reference validation.
- **Placement rules** — a query about where a construct may appear in the grammar. The answer is not expressed in prose but encoded in **grix** admissibility tables. The navigation path follows: concept → grammar role → **grix** matrix → admissibility constraints.

Both cases demonstrate the same property as the “record immutability” trace: the system reaches the answer through structural dependencies rather than lexical similarity.

9.4 Why this is determinism, not guessing

The pipeline is deterministic because every decision is a function of compiled indices:

- enrichment is bounded by T&D and extracted co-occurrence maps (`trma` + Pass 2 artifacts)
- candidate generation is produced by explicit reading programs (`seq[]`) in unit indices
- routing is governed by the concept $\times$ role matrix (`onma`)
- expansion is governed by explicit edges (`xref`)
- extraction is governed by stable coordinates (`linear` chain $\rightarrow$ file $\rightarrow$ span)

The system does not rely on probabilistic prompt exploration. Instead, the system follows a reproducible navigation graph.

This contrasts with generic embedding-based RAG systems, which attempt to retrieve passages that are *semantically similar* to the query. In the “record immutability” example, an embedding system might retrieve sections discussing records in general, but it would have no reliable way to connect the property “immutability” to metadata flags or annex constraints unless that annex happened to contain the same word.

The index topology explicitly encodes the relationship:

record $\rightarrow$ derived type $\rightarrow$ metadata flags $\rightarrow$ immutability rule

The answer is therefore reached through structured inference over the document graph rather than statistical similarity.

A useful mental model is deductive chaining:

If a record is a derived type, and derived types are governed by metadata flags, and metadata flags determine immutability constraints in an annex, then the answer must be retrieved from that annex.

This is the key property: when lexical matches fail, **topology still holds**.

Optional visualization: For publication, the “record immutability” trace can be shown as a compact step diagram (query $\rightarrow$ enrichment $\rightarrow$ indices hit $\rightarrow$ chains $\rightarrow$ annex). A detailed trace (including concrete chains and resolved spans) can be placed in an appendix.

10. Discussion

10.1 Tooling > Fine-Tuning

The model did not change. The navigation layer changed. This is not a minor implementation detail—it is a fundamental shift in how the field approaches domain adaptation for large language models.

Fine-tuning and retraining have long been the default answer to “how to make LLMs expert in domain X?” This work demonstrates an alternative: **if the domain knowledge is already encoded in a structured document, the correct interface is navigation, not memorization.**

Consider the economics:

- **Fine-tuning** requires: curated datasets, compute resources, expertise in model training, and repeated cycles as the document evolves.
- **Structured indexing** requires: one-time compilation of the document (the 3-pass compiler), after which the indices can be reused across any frontier model without modification.

The 500-page specification used in this study was compiled once. The resulting indices have been used with multiple model versions (Claude 3.5 Sonnet, Claude Sonnet 4, GPT-4) with identical accuracy improvements.

No retraining. No fine-tuning. No prompt engineering.

This points to a decoupling of **domain expertise** from **model weights**. Expertise resides in the structure; the model merely reads it. This has practical implications for regulatory AI, compliance automation, and enterprise knowledge management: organizations can treat normative documents as executable assets rather than as training data.

10.2 Prompt Engineering Becomes Secondary

When structure is canonicalized, normative signals are extracted, and linkage chains are explicit, the prompt reduces to:

```
keywords + ontological_role
```

Five parameters. Not five paragraphs.

This is not an aesthetic preference—it is a **stability guarantee**. Every variable introduced into a prompt is a potential source of variance. The more instructions one gives the model about *how* to navigate, the more one relies on its probabilistic interpretation of those instructions. The system presented here offloads the "how" entirely to deterministic tooling.

The model's task becomes trivial:

- 1. Receive a small set of highly relevant fragments (3–5k tokens).
- 2. Read them.
- 3. Answer the question.

No chain-of-thought about where to look. No self-correction loops. No "let me check if I missed something." The navigation is done before the model sees a single token.

This has implications beyond convenience:

- **Reproducibility:** The same query returns the same fragments, leading to the same answer.
- **Auditability:** The reading plan can be logged and inspected.
- **Cost predictability:** Token usage is bounded and stable.

In environments where every answer must be justified (compliance, legal, regulated industries), this is not a nice-to-have—it is a requirement.

10.3 A New Division of Labor

What emerges is a clean separation of concerns:

Layer	Responsibility	Implemented By
Knowledge encoding	Capturing human expertise in stable form	Domain experts writing E.L.I.A. specifications
Navigation	Finding the right fragments for a query	Index compiler + MCP server

Layer	Responsibility	Implemented By
Inference	Reading and reasoning over fragments	LLM

Each layer does what it does best:

- Humans are good at defining meaning, but bad at memorizing 500-page documents.
- Deterministic algorithms are good at indexing and retrieval, but bad at reasoning.
- LLMs are good at reading and synthesizing, but bad at navigating large spaces without drift.

The system respects these strengths and weaknesses. It does not ask the LLM to navigate. It does not ask the index to reason. It does not ask the human to memorize.

This division of labor is the architectural insight behind the results reported in this paper.

10.4 Beyond Specifications: A General Pattern

While this work focuses on ISO-grade normative documents, the pattern is general:

For any domain where knowledge is codified in structured form, deterministic navigation over that structure can outperform probabilistic memorization.

The same approach could apply to:

- Legal code and regulations
- Technical manuals and maintenance procedures
- Medical guidelines and protocols
- Financial compliance rules
- Corporate policy documents

For regulatory and compliance domains specifically, the abstract framework includes a **physical realization index** (*physa*) that maps abstract obligations to concrete implementation surfaces — records, controls, logs, evidence artifacts. In the E.L.I.A. domain, this role is partially filled by *phya*, but in a GDPR or DORA deployment, *physa* would index evidentiary objects and operational controls separately from abstract definitions.

Wherever documents expose hierarchy, cross-reference, and normative constraints, there is an opportunity to build a navigation layer. The model becomes a **structured reader** rather than a probabilistic guesser.

10.5 The E.L.I.A. and ARC Context

This work is part of a broader research initiative: **E.L.I.A. (Enforcement-Level Integrated Architecture)** — a meaning-first formal language and compiler ecosystem designed to enable the transfer of human knowledge into machine-executable form without model retraining. In E.L.I.A., meaning is treated as a **first-class, versioned artifact**. Every semantic construct is expressed in *past, closed form* through a *definition* block that binds name, explanation, enforcement scope, and normative constraints:

```
definition:
  name: "<concept>"
```



```
explained as: "<the meaning – the only source of semantic truth>"
enforced: [<aspects where this meaning applies>]
constraints:
  MUST: "<non-negotiable rules>"
  SHOULD: "<recommended practices>"
  MAY: "<optional behaviors>"
```

This structure mirrors ISO-grade normative logic (RFC 2119) but makes it machine-readable and enforceable. A definition may exist without behavior—but **behavior may not exist without definition**. This inversion ensures that every action taken by an LLM is grounded in explicitly stated human knowledge, not in the model’s statistical approximations.

E.L.I.A. specifications are processed by the **Auto-Regressive Compiler (ARC)**, which attaches formally defined logical structures—domains, modules, enforcement graphs—to language models, constraining and guiding inference without retraining. The index architecture described in this paper is the navigation layer of that compiler: E.L.I.A. provides the *meaning*, ARC provides the *runtime*, and the indices provide the *map*. The skill does not live in the weights—it lives in the structure.

11. Limitations

The presented architecture achieves strong results on a 500-page normative specification, but its applicability and performance are subject to several constraints. Acknowledging these boundaries clarifies where the method excels and where further work is needed.

11.1 Structural Discipline of the Source Document

The system assumes that the input document follows **ISO-grade structural conventions**:

- a stable hierarchy of headings (H1–H6 or equivalent),
- numbered sections and clauses,
- lettered paragraphs or explicit enumeration,
- tables with captions,
- explicit cross-references (e.g., “see §2.7.4”),
- normative keywords (RFC 2119 or similar deontic language).

When these features are absent—as in unstructured blog posts, narrative prose, or documents without a consistent outline—the compiler cannot derive reliable chain addresses or semantic units. For such content, the approach does not apply.

Even within structured documents, **idiosyncratic formatting** (e.g., headings embedded in tables, references written as “cf. section 2.7.4” without a machine-detectable pattern) may require adjustments to the structural adapter (`structure_map.json` and `patterns.json`). While the adapter layer makes the compiler portable, it does not eliminate the need for manual pattern engineering when faced with radically different document grammars.

11.2 Dependency on Accurate Canonicalization (Pass 1)

The entire index system rests on a **deterministic, error-free canonicalization** of the source text during Pass 1. Every heading, paragraph label, and reference must be normalized into a stable chain address

(e.g., 2.7.4/A.). If the parser misinterprets a heading level, fails to detect a lettered paragraph, or mis-resolves a cross-reference, the resulting indices will be incomplete or incorrect.

In practice, the compiler's pattern-based parser achieves high accuracy on well-formed markdown, but it is not immune to formatting edge cases (e.g., nested lists that mimic lettered paragraphs, missing spaces after section numbers). Such errors propagate through the pipeline and can cause retrieval failures. Robustness improvements, such as statistical validation of extracted addresses or human-in-the-loop correction, remain future work.

11.3 Need for Aspect Indices (Unit-Based Indexes)

The current pipeline assumes that **aspect indices** (*phya*, *sema*, *ont*, *desa*, *grma*, etc.) are either already available or can be generated automatically from the document structure. In the E.L.I.A. case, these indices were produced by the compiler's Pass 2, which groups sections by their semantic role (definition, constraint, procedure, etc.). For a new document that lacks explicit semantic markers, an **additional preprocessing step** may be required:

- a human expert could annotate sections with their roles,
- or an AI-assisted phase could propose role assignments based on section titles, introductory phrases, and keyword heuristics.

Once the aspect indices exist, the remaining algorithm (keyword enrichment → index search → reading-plan funnel) operates unchanged. This means the method is **transferable** to any structured document, but the effort of creating aspect indices varies depending on the document's self-description.

Curation economics. The initial creation of aspect indices requires a domain expert who understands the normative structure of the target document type. In practice, this amounts to approximately 20–30 hours of analyst work per document type (at a typical rate of \$25/hour, this is \$500–\$750 per document class — not per individual document). Once the index schema is established for a document type (e.g., “EU regulation” or “ISO language specification”), new documents of the same type can be indexed with significantly less effort because the aspect structure, keyword patterns, and ontological roles transfer across instances. The knowledge base is cumulative: each indexed document enriches the patterns available for the next. Furthermore, the self-healing index evolution path described in Section 11.11 is designed to progressively automate this process — the system observes which index patterns produce effective retrieval and proposes refinements, reducing expert involvement over time. Seven of the eleven unit-based indices in the E.L.I.A. deployment were initially created manually to establish the patterns; the remaining four are already pipeline-generated. The roadmap targets full automation of index creation for known document types.

11.4 Limited Empirical Validation

The accuracy figures reported in Section 9 are based on a **single specification (E.L.I.A.)** and one external regulatory text (DORA). Although the benchmark includes 246 diverse questions and the results are stable across multiple runs, broader validation on other ISO/IEC standards, EU directives, or technical manuals would strengthen confidence in the method's generalizability. Performance may vary with document length, structural complexity, or the density of cross-references.

A mitigating factor is that the E.L.I.A. specification was deliberately written in **standard ISO/IEC document format**: hierarchical numbered sections, MUST/SHOULD/MAY keyword discipline (RFC 2119), normative appendices (Annex A–G), and a formal Terms & Definitions table. This is the same structural convention used

by ISO 27001, IEC 62443, DORA, and hundreds of other regulatory and technical standards. The navigation architecture — aspect indices, ontological routing, chain addressing — operates on these structural features, not on E.L.I.A.-specific content. Consequently, any document conforming to the ISO/IEC drafting conventions is a candidate for the same pipeline without architectural changes.

Furthermore, the system includes a **structure map** (`structure_map.json`) — a declarative adapter that describes the heading hierarchy, section-numbering convention, paragraph markers, and annex structure of a given document type. The structure map is authored once per document class (e.g., "ISO language specification", "EU regulation", "NIST framework") as part of the initial curation phase. When onboarding a new document of the same class, the existing structure map is reused directly; for a new document class, creating the structure map is a bounded task that formalizes the structural conventions the curator already understands. This mechanism makes the algorithm explicitly tunable to arbitrary document formats without modifying the core pipeline.

11.5 Absence of a RAG Baseline Comparison

This work does not include a formal comparison against a Retrieval-Augmented Generation (RAG) baseline. This omission is deliberate and stems from the thesis itself: *domain expertise in LLMs can be achieved without model training by encoding expert navigation strategy into deterministic tooling*. The claim is not "better retrieval" but "no retrieval at all" in the embedding-similarity sense.

RAG and the present system operate in **categorically different paradigms**. A standard RAG pipeline — document → chunks → embeddings → similarity search → top-K → LLM — treats the document as an opaque bag of text fragments. The unit of retrieval is a **chunk**: a fixed-size substring whose boundaries are determined by token count, not by semantic structure. Similarity is measured in embedding space, which captures lexical and topical proximity but is blind to normative modality (MUST vs. MAY), hierarchical position (definition vs. example), and cross-reference topology.

The present system operates as a **compiler**: document → structural analysis → typed indices → deterministic navigation → precise extraction → LLM. The unit of retrieval is a **chain address**: a semantically meaningful fragment (a lettered paragraph, a table row, a section range) whose boundaries are defined by the document's own structure. Navigation is not similarity-based; it is ontologically routed (WHAT/WHY/HOW/WHEN), normatively weighted (RFC 2119 modality signals), and tier-prioritized (definitions before examples before cross-references).

A naive RAG baseline on a normative specification would be a **strawman**. Chunking destroys cross-references (a chunk boundary may fall between a MUST clause and its exception). Embedding similarity cannot distinguish a section that *defines* "delegate" from one that merely *mentions* it. Ontological routing — answering "what is X?" differently from "how to declare X?" — is impossible without structural metadata that embeddings do not encode. Publishing a comparison against such a baseline would demonstrate a predictable failure mode, not a meaningful evaluation.

Conversely, an **enhanced RAG** system — one with structure-aware chunking, normative metadata injection, cross-reference graph traversal, and section-type classification — would converge toward the architecture described in this paper. The enhancements required to make RAG competitive on normative documents are precisely the components I built: the aspect indices encode section types and ontological roles; the chain addressing system provides structure-aware "chunking"; the NDP pipeline injects normative weights. At the limit, enhanced RAG and the present approach are not competitors but points on the same design gradient, with the present system representing the fully compiled endpoint.

That said, a controlled **ablation study** — comparing the full index pipeline against a RAG system using the same document corpus, with identical questions and scoring rubric — would quantify the contribution of each architectural component (ontological routing, normative weighting, chain addressing) relative to embedding-based retrieval. This remains an explicit item of future work.

11.6 The Model Still Must Reason

The architecture eliminates **navigation overhead**, but it does not eliminate the need for the LLM to **read and synthesize** the retrieved fragments. If the extracted content itself is ambiguous, contradictory, or poorly written, the model may still produce incorrect answers. The system guarantees that the right fragments are delivered, but it cannot guarantee that the model interprets them correctly. In practice, I observed that frontier models (Claude 3.5 Sonnet through Sonnet 4, GPT-4) are highly reliable at answering questions once given the exact relevant text, but this may not hold for weaker models or for questions that require complex inference across multiple fragments.

11.7 Embedding Model Ceiling

The MCP server uses a lightweight embedding model ([Xenova/all-MiniLM-L6-v2](#), 384-dimensional, ~22 MB) for two purposes: (1) a **relevance gate** that filters guaranteed noise from Tier-3/4 chains before extraction, and (2) the [analyze_question](#) tool that matches questions against unit description vectors.

MiniLM operates with fixed thresholds (REJECT = 0.19, ACCEPT = 0.65). Below REJECT, a chain is discarded as noise; above ACCEPT, it is promoted as a direct match; between the two thresholds, index keyword matching has the final say. This gating is effective for well-formed specification queries but has known limitations: (a) the model's general-domain vocabulary may not capture highly specialized terms (e.g., "immutability" as a metadata constraint), and (b) the 384-dimensional space provides limited separation in the ambiguity zone near the REJECT threshold. In such cases, the system falls back to structural keyword matching, which is resilient by design.

11.8 Preprocessing Overhead for Non-Markdown Sources

The compiler expects the input document to be in **markdown** with a reasonably clean structure. For documents available only as PDF, Word, or HTML, an additional conversion step is necessary. While tools like [pandoc](#) or [pdf-to-markdown](#) can automate this, the conversion may introduce formatting artifacts that interfere with canonicalization. A production deployment would need a robust document-ingestion pipeline that normalizes various formats into the markdown expected by Pass 1.

11.9 Document Quality Is Not Remediated

The indices reflect the document as written. If the specification contains **internal contradictions, omissions, or ambiguities**, the retrieval system will faithfully return those flawed passages. The method does not perform semantic repair; it only makes existing content addressable. In regulated domains, this is actually a feature (it preserves the authoritative source), but it means that the system's answers are only as good as the document itself.

11.10 Scalability to Extremely Large Corpora

The current implementation handles a 500-page document comfortably: the indices are compact (the [linear](#) index is ~95k lines) and query latency is ~3 seconds. For collections of multiple documents or

specifications running to thousands of pages, the index size would grow linearly, and the search over unit indices might require optimisation (e.g., sharding, more selective keyword matching). However, the fundamental approach—pre-compiled structural indices plus deterministic retrieval—is designed to scale; no component relies on brute-force similarity search over the entire text.

11.11 Index Maintenance and Recompilation

When the source specification is updated, the indices must be recompiled. The current compiler supports **full recompilation** only: all 14 indices are regenerated from scratch. For the E.L.I.A. specification (≈ 500 pages), full recompilation takes approximately 2–3 minutes on commodity hardware.

Incremental recompilation (updating only affected indices when a single section changes) is not yet implemented. In practice, specification edits are infrequent (the E.L.I.A. spec averages 1–2 releases per month), so full recompilation is acceptable. For documents with higher edit frequency, incremental index updates would reduce maintenance overhead and are planned as future work.

For incremental updates, the system includes a maintenance instruction protocol defining deterministic operations: scan modified sections, classify changes, add or update units and chains, validate index integrity, and optionally rebuild derived indices. This protocol has been used in practice for E.L.I.A. specification edits but is not yet automated into a single-command pipeline.

Toward self-healing indices. The current maintenance model is semi-manual: an expert decides which units to add, which chains to update. However, the broader ARC architecture (Section 1, 2) provides a path toward automatic index evolution. The E.L.I.A. specification itself defines a three-layer compiler architecture: a **Classical Compiler Layer** (deterministic validation), an **Auto-Regressive Layer** (probabilistic projection), and a **Self-Modification Layer** (adaptive feedback). The third layer is designed to observe runtime behavior—query patterns, retrieval success rates, chain utilization statistics—and feed this evidence back into the compilation substrate.

The specification's semantic model (§4.2) formally embeds this evolutionary lifecycle through three mechanisms: (a) **controlled semantic degradation**, where constructs lose applicability unless reaffirmed; (b) **bounded experimental exploration** using surrogates as transient carriers of intermediate state; and (c) **evidence-driven natural selection**, where newly formed semantic chains are continuously evaluated against observed facts (admissibility outcomes, emitted events, statistical measures of correctness and usefulness). Accumulated evidence may promote an experimental construct to stable status, require refinement, or allow it to naturally decay.

Applied to the index architecture, this means the Self-Modification Layer can: track which index units are consistently selected vs. skipped; identify keyword gaps (queries that fail enrichment); detect chain addresses that are frequently resolved but never used in final answers; and observe which ontological routing paths produce high-quality results. This telemetry feeds a regeneration pipeline that adjusts unit keywords, reweights coupling pairs, proposes new units for uncovered query patterns, and prunes dead chains—all under the deterministic governance of the Classical Compiler Layer, which validates that regenerated indices preserve structural integrity and normative correctness.

Surrogates (§1.4.6) play a specific role in this feedback loop: they serve as compiler-managed, non-authoritative carriers that preserve enforcement continuity across the adaptive cycle. A surrogate can hold a proposed index modification through validation phases without it acquiring semantic authority. Only after the proposed change passes admissibility checks and demonstrates empirical benefit does it transition to

canonical status. This prevents the adaptive layer from introducing silent semantic drift into the index substrate.

This self-healing capability is not yet implemented in the current MCP server. However, the language-level machinery for it—surrogates, evidence-based selection, the three-layer ARC model—is already specified and forms the intended evolution path from manual curation toward an auto-regressive index maintenance cycle.

11.12 Summary

The method is **powerful within its niche**: ISO-style normative documents that can be parsed into stable chain addresses and enriched with aspect indices. Outside this niche—unstructured content, documents with erratic formatting, or sources lacking normative keywords—the architecture either requires significant adaptation or is not applicable. These limitations are not weaknesses of the approach but rather delineations of its intended use. For the class of documents it targets, the results demonstrate that structured navigation can outperform both generic RAG and whole-document prompting, without any modification to the underlying language model.

12. Related Work

The approach presented here sits at the intersection of several active research areas. I position it relative to each, focusing on the structural distinctions that make deterministic navigation fundamentally different from existing paradigms.

12.1 Retrieval-Augmented Generation (RAG)

Lewis et al. (2020) introduced RAG as a general framework for combining retrieval with generation. Most RAG implementations use dense passage retrieval (DPR; Karpukhin et al., 2020) or learned sparse representations (ColBERT; Khattab & Zaharia, 2020; SPLADE; Formal et al., 2021). These systems retrieve by **semantic similarity** over chunk embeddings, which works well for natural-language corpora but struggles with normative documents where meaning is encoded structurally (section numbering, cross-references, admissibility matrices) rather than lexically.

The core limitations of RAG in this domain are threefold:

1. **Chunk boundary problem.** RAG splits documents into fixed-size or sentence-level chunks. In normative specifications, a single concept may span a heading, three lettered paragraphs, a constraint table, and an annex entry — each at different positions in the document. Chunking destroys these relationships. My system uses chain addresses that respect the document's declared structure (2.7.4/A., 2.7.4/table-1, A.6.2) and can extract precisely the fragments needed, from a single paragraph to a full subtree.
2. **Relevance vs. authority.** RAG ranks chunks by embedding similarity, which conflates *relevance* (the chunk mentions the topic) with *authority* (the chunk is the normative source for the topic). A definition in §2.5.3 and a passing mention in §8.1 may have similar embeddings, but only the former is authoritative. My system encodes authority explicitly through index units with tiered reading sequences — definitions before constraints before examples — ensuring the model reads the authoritative source first.
3. **No ontological disambiguation.** When a user asks "what is a delegate trust boundary?", RAG retrieves chunks containing *delegate* and *trust boundary* independently. It has no mechanism to prioritize the

specific semantic relationship between these two concepts. My ontology funnel classifies the query role (WHAT/WHY/HOW/WHEN/WHERE) and routes through indices that capture the correct relationship, not merely the correct vocabulary.

The system replaces embedding-based retrieval entirely with pre-compiled structural indices and deterministic scoring.

12.2 Knowledge Graph Question Answering

Knowledge Graph QA (KGQA) systems such as EmbedKGQA (Saxena et al., 2020), GraftNet (Sun et al., 2018), and more recently UniKGQA (Jiang et al., 2023) answer natural language questions by traversing entity-relationship graphs derived from knowledge bases (Freebase, Wikidata, DBpedia). Microsoft's GraphRAG (Edge et al., 2024) extends this to narrative documents by building entity-relationship graphs from unstructured text using LLM-based entity extraction and community detection.

Three structural differences separate my approach from KGQA:

1. **Explicit vs. inferred graphs.** KGQA and GraphRAG must *extract* entities and relations from text — an inherently lossy, probabilistic process. Normative documents already declare their structure: numbered sections, lettered paragraphs, explicit cross-references ("see §2.7.4"), and annex mappings. My compiler captures this declared structure deterministically — no entity extraction, no relation inference, no hallucination risk in the graph itself.
2. **Units vs. triples.** KGQA operates over (subject, predicate, object) triples. My system operates over **units** — structured reading programs that encode not just what to read, but in what order, at what budget priority, and for which ontological role. A unit is richer than a triple: it carries keyword surfaces, tiered chain sequences, intent classification, abstracts, and cross-references.
3. **Scoring and budget control.** KGQA traverses graph paths and scores by path length or embedding proximity. My system uses a two-level deterministic scoring function (unit-level keyword matching → chain-level priority scoring with 10+ weighted factors) that incorporates document-specific signals such as territory penalties, heading-pair coupling, and normative modality. Additionally, the tier-based budget trimming mechanism has no analogue in graph QA — it allows graceful degradation under token constraints while preserving authoritative content.

The [xref](#) cross-reference index is superficially similar to a knowledge graph, but it serves a different purpose: it is a traversal aid within an already-structured document, not an extracted approximation of unstructured text.

12.3 Semantic Search

Semantic search encompasses dense retrieval (DPR, Contriever; Izacard et al., 2022), learned sparse methods (SPLADE; Formal et al., 2021), and hybrid approaches that combine BM25 with neural re-ranking (Nogueira & Cho, 2019). These systems map queries and documents into a shared embedding space and retrieve by vector similarity.

The fundamental limitation of semantic search for normative documents is the **semantic gap between query language and specification language**. Consider a hypothetical regulatory specification (e.g., a GDPR-derived compliance document): a user asking "when must I report a data breach?" uses informal vocabulary, while the specification distributes the answer across "notification obligation", "supervisory authority", and "72-hour

window" in separate, cross-referenced sections. In the index architecture, this query would be classified under the WHEN ontological role — a conditional/temporal trigger — routing it through lifecycle and obligation indices rather than through definition or syntax indices. The "when must I" phrasing signals a time-bound normative constraint, which deterministic routing resolves to the correct regulatory clause. Embedding similarity may bridge some of this gap, but it cannot:

1. **Distinguish identical vocabulary across roles.** The word "record" appears in type definitions (WHAT), design rationale (WHY), syntax blocks (HOW), and temporal rules (WHEN). Semantic search retrieves all occurrences indiscriminately. My system uses ontological routing to deliver the correct role-specific fragments.
2. **Respect hierarchical authority.** A definition in a Terms & Definitions table and a usage example in an annex may be equally "similar" to the query embedding. Semantic search cannot express that one is normatively authoritative and the other is illustrative. My index architecture encodes this distinction through tiers and intent labels.
3. **Support structural extraction.** Semantic search returns document chunks (typically 256–512 tokens). My system returns chain-addressed fragments that can be a single paragraph, a lettered range, a table, or a full recursive subtree — all derived from the document's declared structure rather than arbitrary windowing.

My system does use full-text search (ripgrep) as the extraction mechanism, but the *routing* that determines what to search for and where is driven by pre-compiled indices, not by embedding similarity.

12.4 Ontology Routing Systems

Ontology-based information retrieval has a long lineage: from early ontology-driven query expansion (Navigli & Velardi, 2003) and the Semantic Web vision (Berners-Lee et al., 2001) to modern ontology-enhanced QA systems (Diefenbach et al., 2018). These systems typically use formal ontologies (OWL, RDF, SKOS) to expand queries, resolve ambiguity, or constrain answer types.

My system shares the *goal* of ontological disambiguation but differs in architecture and scope:

1. **Lightweight, document-derived ontology.** Classical ontology routing requires a pre-existing formal ontology (e.g., a medical ontology for clinical QA, a legal ontology for regulatory QA). My system derives its ontological structure directly from the document being navigated — the five roles (WHAT/WHY/HOW/WHEN/WHERE) are grounded in the document's own structural conventions (definitions, rationale sections, syntax blocks, lifecycle sections, admissibility matrices). No external ontology needs to be authored, maintained, or aligned.
2. **Coupling pairs instead of subsumption hierarchies.** Traditional ontologies use is-a, part-of, and subsumption relations. My system uses **semantic coupling pairs** — explicit multi-role bindings (HOW×WHEN, WHAT×WHY, WHERE×HOW) attached to index units — that express which question types a unit can answer simultaneously. This is a pragmatic navigation mechanism, not a formal knowledge representation.
3. **Compiled indices, not runtime inference.** Ontology-based systems typically perform runtime reasoning (description logic inference, SPARQL queries, ontology matching) to route queries. My system pre-compiles all routing decisions into index artifacts at build time. At query time, routing is a

deterministic lookup — scoring and sorting, not inference. This eliminates the latency and non-determinism of runtime ontology reasoning.

4. **Integration with budget and normative control.** Classical ontology routing produces candidate answers but does not manage token budgets, tier-based trimming, or normative re-ranking (MUST/SHOULD/MAY). My system unifies ontological routing with reading plan construction, budget management, and normative-driven pipeline — a full navigation stack rather than a standalone disambiguation layer.

12.5 Structured Document Understanding

LayoutLM (Xu et al., 2020) and DocQA systems process visual document layouts. These target scanned/PDF documents where structure must be *inferred* from spatial layout. My approach assumes the document is already in structured markdown with explicit hierarchy — a common case for ISO/IEC, EU, and RFC standards. The two approaches are complementary: LayoutLM could serve as a preprocessing stage to recover structure from scanned documents, which my compiler then indexes.

12.6 Tool-Augmented LLM Agents

ReAct (Yao et al., 2023) and Toolformer (Schick et al., 2023) equip LLMs with tools and let the model decide when and how to use them. My system differs in two ways: (1) retrieval is deterministic rather than model-directed — the model does not decide *what* to search, and (2) the navigation logic is pre-compiled into indices rather than learned at inference time. This eliminates the variance inherent in agentic tool loops. The benchmark comparison (Section 11) quantifies this: the agentic baseline uses 7× more tokens and exhibits measurable accuracy variance, while my system produces identical results across runs.

12.7 Domain-Specific Fine-Tuning

LoRA (Hu et al., 2022) and related parameter-efficient methods inject domain knowledge into model weights. My work demonstrates that for structured normative documents, comparable accuracy can be achieved at inference time via navigation tooling, without any weight modification. The two approaches are complementary: fine-tuning adapts the model's *reasoning style*, while my system adapts its *information access*. Notably, the capability amplification effect (Section 11.4) — where a lower-capability model achieves frontier-level accuracy when equipped with deterministic navigation — suggests that navigation, not model capacity, is the binding constraint for specification reasoning.

13. Conclusion

I did not teach the model new knowledge.

I taught it how to read.

The skill does not live in the weights.

It lives in the structure.

By compiling that structure into a deterministic index, the system gains precise control over the model's context window—delivering only the relevant fragments (2-20k tokens, avg ~12k) instead of forcing it to scan hundreds of pages. This control is what makes specification reading both **affordable** (up to 15× fewer spec tokens vs full-context, 7× fewer total tokens vs agentic baseline) and **reliable** (100% accuracy, zero variance). Navigation overhead, the dominant factor in naive LLM comprehension, is eliminated.

A note on experimental validity: The benchmark used the E.L.I.A. specification—a language developed as part of this research. This document is **guaranteed not to appear in any LLM training corpus**. No web crawl, no GitHub leak, no accidental inclusion. This eliminates the risk of data contamination and ensures that the model's success is attributable solely to the navigation architecture, not to memorization from pre-training. The experiment measures genuine capability transfer, not retrieval from hidden memory.

Every ISO/EU normative document already contains the latent structure required for deterministic AI interpretation. The compiler does not invent meaning; it merely makes that structure executable. The implication is clear: for structured domains, tooling outperforms training. The future of domain-specific AI lies not in larger models, but in smarter navigation of the knowledge that already exists.

References

S. Bradner. Key words for use in RFCs to Indicate Requirement Levels. RFC 2119, IETF, 1997. URL: <https://tools.ietf.org/html/rfc2119>

ISO/IEC. *ISO/IEC Directives, Part 2: Principles and rules for the structure and drafting of ISO and IEC documents*. 9th edition, 2021.

P. Lewis, E. Perez, A. Piktus et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

V. Karpukhin, B. Oğuz, S. Min et al. Dense Passage Retrieval for Open-Domain Question Answering. *Proceedings of EMNLP*, 2020.

D. Edge, H. Trinh, N. Cheng et al. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *arXiv:2404.16130*, 2024.

S. Yao, J. Zhao, D. Yu et al. ReAct: Synergizing Reasoning and Acting in Language Models. *Proceedings of ICLR*, 2023.

T. Schick, J. Dwivedi-Yu, R. Dessì et al. Toolformer: Language Models Can Teach Themselves to Use Tools. *Advances in Neural Information Processing Systems*, 2023.

E. J. Hu, Y. Shen, P. Wallis et al. LoRA: Low-Rank Adaptation of Large Language Models. *Proceedings of ICLR*, 2022.

Y. Xu, M. Li, L. Cui et al. LayoutLM: Pre-training of Text and Layout for Document Image Understanding. *Proceedings of KDD*, 2020.

N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, P. Liang. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. (Presented at NeurIPS 2023 Workshop.)

O. Khattab, M. Zaharia. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. *Proceedings of SIGIR*, 2020.

T. Formal, B. Piwowarski, S. Clinchant. SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking. *Proceedings of SIGIR*, 2021.

A. Saxena, A. Tripathi, P. Talukdar. Improving Multi-hop Question Answering over Knowledge Graphs using Knowledge Base Embeddings. *Proceedings of ACL*, 2020.

H. Sun, B. Dhingra, M. Zaheer, K. Mazaitis, R. Salakhutdinov, W. Cohen. Open Domain Question Answering Using Early Fusion of Knowledge Bases and Text. Proceedings of EMNLP, 2018.

J. Jiang, K. Zhou, W. X. Zhao, J.-R. Wen. UniKGQA: Unified Retrieval and Reasoning for Solving Multi-hop Question Answering Over Knowledge Graph. Proceedings of ICLR, 2023.

G. Izacard, M. Caron, L. Hosseini et al. Unsupervised Dense Information Retrieval with Contrastive Learning. Transactions on Machine Learning Research, 2022.

R. Nogueira, K. Cho. Passage Re-ranking with BERT. arXiv:1901.04085, 2019.

R. Navigli, P. Velardi. An Analysis of Ontology-based Query Expansion Strategies. Proceedings of the Workshop on Adaptive Text Extraction and Mining (ATEM), 2003.

T. Berners-Lee, J. Hendler, O. Lassila. The Semantic Web. Scientific American, 284(5):34–43, 2001.

D. Diefenbach, V. Lopez, K. Singh, P. Maret. Core Techniques of Question Answering Systems over Knowledge Bases: A Survey. Knowledge and Information Systems, 55(3):529–569, 2018.

ANNEX A — Real MCP Trace Dumps

Three unedited diagnostic traces from the MCP spec-reader server (v0.3.0, **verbose: debug**) are provided as supplementary material. They demonstrate the deterministic pipeline on queries of varying complexity:

1. **"record immutability"** (WHAT) — keyword enrichment yields 2 terms; 3 units matched; 257 chains considered, 107 rejected by relevance gate; answer reached in \$2.5/\$A.12 despite no heading containing "immutability".
2. **"delegate trust boundary, cross-border enforcement"** (HOW) — 7 enrichment terms extracted from normative paragraphs; 248 chains reduced to 84 after 66% deduplication; routes to \$7.0–\$7.5.
3. **"action syntax, action body"** (HOW-Code) — NDP processes 58 normative paragraphs yielding 18 enrich + 15 filter + 5 conditional keywords; compact 20-chain plan reaches \$2.6.5.5 (3 levels deep) with full AST grammar.

Full traces with all intermediate decisions available at: [annex_a_trace_dumps.md](#)

ANNEX B — Test Methodology

The evaluation methodology, scoring rubric, and per-question results are documented in full as supplementary material. Key parameters:

- **Question sets:** 246 full benchmark + 36 hard subset + 10 conceptual questions, all authored before any runs.
- **Four evaluation modes:** MCP (full system), Manual (grep + read_file), Naive (full spec in context, ~178k tokens), Conceptual (4-artifact interactive protocol with plan/render/final/bench cards).
- **Scoring:** 3-axis rubric (Accuracy 0–5, Completeness 0–5, Noise 0–5), composite quality = (Acc×2 + Comp – Noise) / 15.
- **Models tested:** Claude Sonnet 4 (primary), Claude Opus 4.6 (extended), Claude Haiku 4.5 (stress test).

- **Controls:** no data contamination (private spec), scorer $\neq$ answerer, raw dumps preserved, 5 $\times$ stability runs per question.
- **Artifact count:** 800+ evaluation artifacts across 3 models, 4 modes, and 13 MCP server versions.

Full protocol, scoring tables, bench cards, and runner code available at: [annex_b_test_methodology.md](#)